

GEORISK-SPARK

Real-Time Geospatial Supply Chain Disruption and Route Intelligence

TEAM PROJECT GUIDE

A practical, four-person Big Data Analytics project plan for a 12-week semester. This guide explains the problem, the system, the work split, and how the team will build and demonstrate a small, repeatable prototype using synthetic data and AWS.

Project promise: turn a stream of vessel positions and marine alerts into cell-based risk events, corridor density measures, delay estimates, and a safer-route demonstration.

The problem

Logistics teams can learn about storms and congestion after vessels are already affected. Repeatedly comparing millions of GPS points with complex hazard polygons is expensive. This project explores an indexed approach: map vessel coordinates and storm coverage into H3 hexagonal cells, then use matching cell IDs to narrow down possible risk events.

What the semester prototype will demonstrate

- A repeatable synthetic feed of vessel telemetry and storm alerts.
- A Spark streaming pipeline that maps events to H3 cells, flags hazard overlap, and measures unique vessels per cell over 15-minute windows.
- An S3 data lake with queryable Parquet results, a delay model, a route alternative, and an interactive map dashboard.
- A cloud-oriented workflow using Kinesis, AWS Glue, S3, the Glue Data Catalog, and Athena, with compute started only for development or demonstrations.

Scope guard

This is an academic proof of concept. It uses generated data, a small route graph, and a demonstration prediction model. It does not provide navigational advice, live vessel tracking, or operationally validated safety forecasts.

1. System architecture

The pipeline keeps raw events, spatial processing, analytics, and presentation as separate responsibilities. During the first milestone it runs locally with sample files. Later milestones add AWS services behind the same event contracts.

INPUTS	STREAM	SPARK + H3	DATA LAKE	USE
Synthetic vessel JSON Weather alert GeoJSON	Kinesis streams telemetry + alerts	AWS Glue Spark streaming validate, index, join, aggregate	S3 Parquet Glue Data Catalog	Athena queries MLlib, routes, dashboard

End-to-end data flow

Stage	What happens	Output
1. Generate	Replay deterministic vessel locations and time-bounded weather alert polygons. Keep event IDs and event time so runs can be repeated and compared.	Versioned JSON events
2. Ingest	Publish telemetry and hazard events to separate Kinesis Data Streams. Preserve the local sample path for development without AWS access.	Two managed streams
3. Index and assess	AWS Glue runs Spark Structured Streaming. Convert vessel positions and polygon coverage to H3 cells, compare cell IDs, expire inactive alerts, and calculate 15-minute density.	Risk events and density rows
4. Store	Write raw and processed results to date-partitioned S3 prefixes in Parquet. Register schema and location in the Glue Data Catalog.	Queryable history
5. Analyze and show	Athena queries historical outputs; Spark MLlib estimates demo delay; a small graph finds a lower-risk path; Streamlit renders map layers and metrics.	Dashboard and demo

Cloud cost rule: streaming jobs keep running and accrue charges while active. Start jobs for a defined test or demo window, inspect the output, then stop them.

2. Four-person ownership and handoffs

Each person owns a layer with a concrete deliverable. Shared interface reviews prevent the team from discovering incompatible schemas at the end of the semester.

Owner	Primary responsibility	Deliverables	Handoff
Member 1 Ingestion + contracts	Create deterministic synthetic vessel and hazard data; validate event format; later publish to Kinesis.	JSONL replay generator, sample events, schema notes, Kinesis producer, data-quality counters.	Agrees field names and timestamp semantics with Members 2 and 3 before streaming integration.
Member 2 Streaming + spatial	Build the Spark Structured Streaming flow, H3 conversion, active-hazard matching, alerts, and windowed density.	AWS Glue job, H3 utility functions, checkpoint settings, processing metrics, output schema.	Consumes Member 1 contracts and writes the risk/density contract Member 3 stores.
Member 3 Storage + analytics	Own S3 layout, Parquet outputs, Glue Catalog/Athena queries, and the MLlib delay baseline.	Storage prefixes, catalog tables, example SQL, model training/evaluation report.	Uses Member 2 outputs and publishes fields/queries for Member 4's dashboard.
Member 4 Routing + dashboard	Model a small port/sea-lane graph, apply risk threshold, calculate alternative path, build Streamlit map and metrics.	Graph fixture, route comparison, dashboard, demo script and integration checklist.	Consumes stable storage/query outputs and returns missing fields to the owners early.

Team working agreement

- Agree the event contracts and output column names by the end of Week 2. Treat changes as a team decision and update the guide and sample fixtures.
- Use one feature branch per focused change. Open a pull request to main, ask one teammate to review it, and keep commits small and descriptive.
- Integrate weekly. At every meeting, demonstrate one runnable path and record blockers with an owner and next action.
- Share responsibility for the final system demo and explanation; each owner presents their layer and its handoff.

A good handoff includes: agreed input/output fields, a runnable example, how to start it, expected output, known limits, and the name of the next owner.

3. Data contracts and geospatial behavior

Keep event payloads small, explicit, and versioned. Use UTC timestamps and GeoJSON's standard longitude-latitude coordinate order for hazard geometry.

Event	Required fields	Example / meaning
Vessel telemetry	schema_version, event_id, event_time, vessel_id, latitude, longitude	schema_version=1; event_time=2026-10-04T09:00:00Z; latitude=37.7749; longitude=-122.4194
Weather alert	schema_version, event_id, alert_id, event_time, valid_from, valid_until, hazard_type, severity, geometry	GeoJSON Polygon; severity 1-5; valid_until expires an active alert.
Risk result	event_time, vessel_id, h3_index, h3_resolution, risk_alert, matching_hazards	One output per valid telemetry event; matching_hazards lists the alert IDs and severities.
Density result	window_start, window_end, h3_index, h3_resolution, unique_vessel_count	15-minute event-time window; deduplicate vessel IDs per cell and window before counting.

H3 workflow

- Use H3 resolution 7 for the first end-to-end cloud demonstration; allow resolution 8 as a finer-grid comparison. Keep resolution on every output row.
- Convert each vessel latitude/longitude to one H3 cell. Convert the storm polygon to the set of H3 cells selected for the chosen resolution.
- Join by cell ID to identify candidate exposure. Attach alert validity and severity so expired hazards do not continue to trigger alerts.
- Treat the match as approximate. A polygon-to-cell method can include or omit partial boundary cells depending on the coverage rule. For production-grade boundary decisions, add exact geometry refinement and validate the chosen H3 overlap mode.

Delay prediction and route scoring

Build a small, reproducible synthetic training set. Define the demonstration delay label from congestion, hazard severity, and a controlled noise term. Compare Spark MLlib Gradient-Boosted Trees against a simple mean-delay baseline using a time-based holdout. In the route graph, score sea-lane edges using travel cost plus a documented risk penalty; remove or heavily penalize edges above the chosen safety threshold and calculate the lowest-cost remaining path. Clearly label these outputs as simulated.

4. Twelve-week delivery plan

The schedule is divided into four three-week phases, with integration checkpoints in every phase. Finish a thin vertical slice before expanding the volume or sophistication of each layer.

Weeks	Focus	Milestone evidence
1-3 Foundation	Confirm contracts, H3 resolution, sample corridor and alert semantics. Initialize GitHub. Create project structure, local sample runner, README, and team PDF.	Fresh clone can run the sample flow and show at least one vessel with a storm-cell match. Repo and docs are usable by all four members.
4-6 Streaming + spatial	Add deterministic replay, Kinesis producers/streams, AWS Glue Spark streaming, H3 hazard coverage, alert expiry, and deduplicated 15-minute density.	A short cloud run consumes events and writes inspectable risk and density output; checkpoint and failure behavior documented.
7-9 Lake + analytics	Write raw and processed Parquet to S3, register tables, query with Athena, train and evaluate the delay baseline, publish query results to the app layer.	Athena returns expected totals and cell-level summaries; held-out model metrics are compared with the mean baseline.
10-12 Route + demo	Finish risk-weighted route alternatives, Streamlit layers, integration, repeatable demo, cost/security check, and course presentation.	One scripted demo shows input, risk event, density, prediction, path comparison, and dashboard; owners can explain limitations.

Weekly cadence

When	Activity
Start of week	Agree each person's deliverable and dependencies. Freeze schema changes unless they are needed to unblock integration.
Midweek	Exchange sample inputs and outputs across adjacent owners. Resolve naming, timestamp, coordinate-order, and partition questions before polishing.
End of week	Run the integrated demo, record what works, update README/guide, and choose the next smallest end-to-end milestone.

If the schedule slips, protect the complete stream-to-risk-to-storage-to-dashboard path. Reduce model sophistication or graph size before dropping integration time.

5. AWS deployment, cost, and safe operation

The first foundation uses local files and does not create cloud resources. Later, build AWS in a student account with least-privilege roles, clear resource names, and an explicit teardown checklist.

Service	Project role	Operating guidance
Amazon Kinesis Data Streams	Separate streams for vessel telemetry and weather alerts.	Use small demo traffic. Record shard/retention configuration and stream names in project setup docs.
AWS Glue streaming	Spark Structured Streaming reads Kinesis, applies H3 risk and density logic, and writes results.	Run only during a test/demo window. Configure checkpoints in S3 and document how to stop the job.
Amazon S3	Raw event archive and processed Parquet data lake.	Use separate raw and processed prefixes. Partition by event date; avoid a partition per H3 cell.
Glue Data Catalog + Athena	Catalog Parquet tables and query results in S3 with SQL.	Use compressed columnar output and bounded sample queries; inspect scanned data volume.
Streamlit dashboard	Display query-backed metrics, vessel positions, hazards, alerts, and detour comparisons.	Keep credentials outside source control. Use local AWS profile/SSO for student setup; do not embed keys in the app.

Security and cost controls

- Never commit access keys, secret tokens, account IDs in public examples, or local AWS config. Use IAM roles and a local named profile or SSO session.
- Use a dedicated project bucket and least-privilege execution role limited to its streams, bucket prefixes, catalog tables, and logs.
- Start long-running streaming jobs only for planned demos. Set an owner and teardown reminder for every paid resource.
- Check current AWS pricing, free-tier eligibility, region availability, and account permissions before provisioning. Free-tier coverage is not assumed for every service or job duration.
- Use synthetic positions and explicitly label all alerts, model outputs, and route advice as demo data.

References

AWS Glue streaming ETL: <https://docs.aws.amazon.com/glue/latest/dg/add-job-streaming.html>

Athena SQL and S3 data catalogs: <https://docs.aws.amazon.com/athena/latest/ug/using-athena-sql.html>

H3 documentation: <https://h3geo.org/docs/>

6. Foundation setup and demo runbook

The repository's first commit is intentionally small. It establishes a common interface and lets teammates validate the spatial idea before adding cloud services.

Current foundation contents

- README and requirements file with Python setup instructions.
- Sample telemetry in JSON Lines and one storm polygon in GeoJSON.
- A local command that validates records, converts vessel coordinates and polygon coverage to H3, and emits one risk result per vessel.
- An .env.example with placeholders only, plus Git ignore rules for credentials, local environments, caches, and generated output.
- This project guide PDF, including team responsibilities, interfaces, schedule, AWS path, runbook, and validation criteria.

Run the local demonstration

Step	Action	Expected result
1	Create a Python environment and install requirements.txt.	H3 Python package is available; no AWS credentials are needed.
2	Run the command below from the repository root.	Three input records are processed and a JSONL output file is created.
3	Inspect output/data/risk-events.jsonl.	Every row has an H3 ID and risk_alert boolean; the two San Francisco fixtures match STORM-SF-001 and the Los Angeles fixture does not.
4	Repeat with --resolution 8.	Cell IDs change to the finer resolution; the output remains valid and the sample risk result remains explainable.

```
python -m pip install -r requirements.txt
python src/georisk/foundation.py --telemetry data/sample/vessel-telemetry.jsonl --hazards data/sample/weather-alerts.geojson --output output/data/risk-events.jsonl
```

Acceptance checks

- Malformed JSON, missing required fields, invalid coordinates, unsupported geometry, or invalid resolution produce a clear error rather than a misleading result.
- The sample output is deterministic and contains no secrets or live personal data.
- The local path runs without AWS. Later cloud milestones add checks for stream consumption, windowed unique-vessel counts, S3/Athena reads, held-out model evaluation, risk-triggered route changes, and dashboard rendering.
- All results communicate that H3 matching is approximate and that predictions and route scores are synthetic demonstrations.

7. Repository workflow, risks, and next steps

Repository workflow

- Use main as the shared integration branch and one short-lived feature branch per cohesive task. Review changes before merging.
- Make focused commits that explain the change, for example: docs: define telemetry and hazard event contracts; feat: add local H3 risk check.
- Push each reviewed milestone so all members can work from the same state. Never force-push main. Do not commit generated local output or credentials.
- The target remote is <https://github.com/saisasidharpaluri/GeoRisk-BDA-Project.git>. Its refs were checked before initialization and the repository had no refs, so the first push can establish main without replacing existing history.

Risks and response

Risk	Response
Cloud access or credits unavailable	Finish the local pipeline and synthetic demo; use AWS resources only after the team confirms account access and estimated cost.
Streaming job integration takes longer than expected	Get one stream-to-H3 output working first; keep local replay as the stable fallback and add the second stream once the first is reliable.
H3 boundary effects confuse risk results	Document coverage semantics, test a point clearly inside and outside a hazard, and add exact geometry refinement only if the course scope permits.
Synthetic data makes model scores look too strong	Use a time-based holdout, publish the mean baseline, disclose how the target was generated, and avoid claiming real-world predictive validity.
Late integration exposes incompatible outputs	Freeze contracts by Week 2, exchange fixtures weekly, and make end-to-end review part of each three-week milestone.

The first three project actions

1. Foundation	Initialize main, add the small local sample flow and documentation, create the project guide PDF, commit and push.
2. Contract review	Have all four members review the event fields, timestamp rules, H3 resolution, and output columns; assign owners and confirm access.
3. First cloud slice	Provision only the minimal AWS resources needed for one short Kinesis-to-Glue-to-S3 demo, then query the result with Athena and tear down unused resources.

A successful semester outcome is a coherent, reproducible pipeline that every teammate can explain - including its assumptions, limits, cloud costs, and the evidence behind its outputs.